Gasballoon Cockpit

A single-file HTML web app for gas balloon flight planning and in-flight monitoring: live position tracking, wind-based landing predictions, staged descent planning, weather stations, air traffic, and offline map caching - built for use on a tablet in the basket.

Current version: v260817.11-1545 (17.08.2026) - this number always matches the `APP_VERSION` constant near the top of the script in `index.html`. Versioning scheme: `vYYMMDD.zz-HHMM` (date of the last change, a 2-digit counter that resets to 01 each new day, and the build time), so multiple same-day builds are unambiguous at a glance - the version is shown in the bottom-left corner of the app itself, useful for confirming a deployment actually picked up the latest build rather than a stale cached one. `cors_test.html`'s own version marker is kept in sync with this.

What it is

One HTML file, no build step, no server. Open it in a browser (or install it to a home screen as a PWA, with `apple-touch-icon.png`) and it runs entirely client-side, calling a handful of free public weather/mapping APIs directly from the browser. A service worker (`sw.js`) caches map *tile* requests only - deliberately not weather/elevation data, which needs to stay fresh - so previously-viewed map areas remain available if the connection drops.

Password gate on load (SHA-256 hashed, set in `GATE_HASH`).

Built for genuine 24/7 unattended operation: the service worker checks for its own updates every 30 minutes (not just on page reload, which might never happen once the app is left running), and a screen wake lock keeps the display on with automatic re-acquisition if the browser ever releases it.

The hamburger menu's last entry, "About - more info!", opens this README as a PDF in a small viewer page with always-visible Print/Close buttons (readme_viewer.html), rather than the browser's native PDF viewer directly.

Zoom-in/out buttons (and the very first automatic centring on GPS after boot) target the visually VISIBLE map area's centre, not the underlying map container's own geometric centre - relevant whenever the sidebar is open, since it overlays on top of the map rather than shrinking it.

Main functions

The app has two main functions, switched via a pill toggle at the top of the map.

Landing Area (default)

Shows the balloon's live predicted trajectory: a cruise segment (green) that transitions into a descent (yellow) at a configurable time ("Initiate descent in"), continuously recalculated as position, altitude, and wind data update. A Monte Carlo scatter of likely landing points is shown as a shaded polygon around the predicted landing point, with a black cross marking its actual centre (computed from the raw scatter itself, not the containing polygon's own vertices, which would skew it toward the edge). The map view gently follows this polygon as it shifts

in response to slider changes - only actually panning/zooming if it would otherwise fall outside the visible area, debounced so dragging a slider doesn't cause the map to jump on every intermediate value.

The descent point (a yellow cross on the trajectory) can be adjusted two ways that stay in sync: the "Initiate descent in" slider (10-minute steps, defaults to 20min), or dragging the cross itself along the trajectory.

Extended trajectory preview: click anywhere within roughly 60° of the trajectory's own direction, beyond where the normal prediction/landing area already reaches, to see the trajectory extended further out (blue) - a frozen snapshot from the moment of the click, useful for comparing the real flown track against an earlier prediction.

A one-time onboarding hint appears the first time this function is used.

Plan Descent

Two sub-functions, switched via a smaller toggle directly under the main "Plan Descent" button.

Quick Descent (default): click any point on the map to find the nearest reachable landing area and the descent path to it - shown as an orange cross (the calculated descent point) and a yellow descent path, same visual language as Landing Area's own live prediction. The reference trajectory truncates at the descent point once a plan is active. The descent point can be dragged to explore alternatives, or a new point clicked anywhere to restart. The reference trajectory itself extends as far as the "Initiate descent in" slider is set to.

Staged Descent: drag a royal-blue cross marker along the trajectory to choose when descent begins, then click within the resulting reachable area to search for a staged descent plan - one that pauses at one or more intermediate altitudes (chosen automatically where wind conditions genuinely differ) rather than descending continuously. Opens a side panel with a chart of the planned stages: altitude, wind at each level, duration, an estimated ballast requirement for each transition, and markers for any wind shear, inversion, or isothermal layers encountered. Minimum stage separation and maximum intermediate stages live directly in this panel. Its own two sliders (max descent time - 10-minute steps, defaults to 20min; inter-stage rate) sit side by side in one row.

Both sub-functions show a one-time onboarding hint on first use. Switching back to Landing Area clears the descent path from the map but leaves the reachable area, Monte-Carlo landing area, and its centre marker in place, with a "Clear staged descent area" button to remove them explicitly.

Flight data box

A small draggable panel showing course (always 3 digits, e.g. 065°), speed (km/h and kn), climb/sink rate, and altitude (m AMSL, ft, flight level above a configurable transition altitude, and AGL). Refreshes on a fixed 0.5s cadence, decoupled from the underlying GPS fix rate. Snaps flush against whichever map edge it's dragged past; defaults to the left edge just below the header button row. Dims along with the map in night mode.

Descent Parameters

"Initiate descent in" and "Descent rate" sit side by side. Descent rate's own track is a wedge shape that grows taller toward higher values, visualizing the accelerating descent speed - the draggable point itself still runs along a flat baseline; only the fill behind it

changes shape. Below each slider: the computed distance/effective rate, plus (for Descent rate specifically) the adiabatic lift bonus as a weight-equivalent ballast figure - the actual extra lift the adiabatic cooling effect provides during descent, typically a few kg. Intercept AGL, Rate below, and Scatter sit in a second row, each with its own computed time-to-reach badge.

Landing Area sidebar card

Shown for whichever landing-point marker is currently the relevant one (Landing Area's live prediction, Quick Descent's planned point, or Staged Descent's), with these fields:

- **Ground wind** at the predicted landing time, with wind gusts (when the model supports them) shown as a second line below it.
- **Terrain near landing point**: elevation-based roughness (flat / moderate slope / steep), nearby tagged buildings or forest, the specific land cover at the exact point (meadow, farmland, forest, residential, water, etc.), and a red "CAUTION WATER!" override if the point falls inside a mapped lake, river, or reservoir - checked via genuine polygon containment, not just proximity. This check runs on its own independent request queue, separate from the app's other Overpass-based features, so a failure elsewhere (airports, roads, place names) can't block it.
- **Sun at estimated landing**: azimuth (always 3 digits) and elevation, with a direction arrow that rotates to point at the sun's actual position, plus remaining time until evening civil twilight (HH:MM) - the more relevant cutoff than sunset itself for whether there's still enough light to see the terrain.
- **Moon phase**: a filled SVG silhouette (not an emoji, for consistent rendering across devices) with illumination percentage and a small waxing/waning trend arrow, plus rise/set times.

Weather modeling

Wind is fetched per-altitude across 19 pressure levels (18 for the two Météo-France models below, which don't publish 975hPa) from the auto-selected model for the current location (ICON-D2 for DACH, ICON-EU for wider Europe, GFS globally), refreshed on a timer (default 15min, configurable). CAPE and wind gusts are fetched as separate, independent requests on a reduced schedule (every 3rd wind refresh) so an unsupported variable on a given model can never take down the core wind fetch.

Besides the auto-selected models, ARPEGE Europe (Météo-France, 11km, all of Europe) and AROME France (Météo-France, 2.5km, France and immediate neighbours) can be picked manually for model comparison - both publish the pressure-level data this app needs. MeteoSwiss's own ICON-CH1/CH2 (1-2km, Switzerland) and the UK Met Office's models were deliberately not added: as of this writing neither publishes pressure-level/wind-at-altitude data through Open-Meteo, which the wind profile this app relies on requires.

Ensemble modeling: with a commercial Open-Meteo API key set (see below), the Monte-Carlo landing-area scatter in all three functions draws on real ensemble model members (ICON-D2-EPS / ICON-EU-EPS / GEFS, ~20-40 members depending on model) instead of an artificial age-based heuristic - each Monte-Carlo sample uses an actual member's own wind profile, genuine forecast uncertainty rather than a random perturbation. A small badge next to the model name in the header ("E" green / "H" gray/amber) shows which mode is currently active. Falls back to the heuristic automatically without a key or if the ensemble fetch fails. Open-Meteo's Ensemble API specifically requires the Professional or Enterprise commercial tier - a Standard/base commercial key (which unlocks the main forecast/elevation/CAPE endpoints just fine) is not enough; the badge

turns amber rather than plain gray in that specific case, distinguishing “no key at all” from “key present but this tier doesn’t include ensemble access”. The account behind this app’s key was upgraded to API Professional on 17.08.2026, confirmed reachable via a live test (HTTP 200). With the resulting higher monthly quota (5M vs 1M calls), the ensemble fetch’s own pressure-level set was also widened from 8 to 11 levels for finer vertical resolution.

Also found and fixed once real data was reachable: Open-Meteo’s ensemble member keys use 2-digit zero-padded numbering (“_member01”, not “_member1”) - the code previously reconstructed keys without that padding, so members 1 through 9 (a majority of a typical 20-40 member ensemble) silently found no matching data and ended up with empty wind profiles, which windAt() then treats as zero wind. This would have severely distorted the scatter even once the API itself became reachable. The padding width is now detected directly from a live response’s own keys rather than assumed.

Power lines

Overhead line towers and transformers are rendered as small, muted markers rather than bright coloured circles, so they don’t visually compete with the lines themselves (able to be followed continuously) - spotting the wires is what actually matters for flight safety, not each individual mast along the route. Towers (neutral grey) and transformers (muted blue-grey) use distinct tones, not identical ones, so toggling one category’s checkbox in Settings while leaving the other on stays visually verifiable rather than looking like filtering stopped working. Category checkboxes and the “overhead only” toggle now discard and fully rebuild the power-lines layer instance from scratch (not just remove/re-add the same one) - a first fix relied on VectorGrid’s own redraw() (documented upstream as unreliable), a second removed and re-added the same instance, and even that turned out insufficient, very likely because VectorGrid keeps its own internal tile cache tied to the instance itself regardless of map attachment. Rebuilding the instance entirely cannot carry over any stale internal state.

Found the actual source of persistent “blue circles and half-circles not connected by any line” that survived all three earlier fix attempts above: OpenInfraMap’s combined tile endpoint bundles several vector layers beyond power - telecoms (including microwave radio links, point-to-point wireless connections with no cable between them, often drawn as isolated markers or directional half-circle antenna-orientation symbols), petroleum, and water infrastructure. Only the power_* layers were ever explicitly styled; any other layer name fell back to VectorGrid’s own default rendering, which is what those blue shapes almost certainly were. The whole style definition is now wrapped in a Proxy so every layer name NOT explicitly listed is hidden by default - robust against not knowing (or OpenInfraMap changing) the exact non-power layer names, rather than needing to enumerate and hide each one individually.

Telecom/antenna masts are still shown, though - unlike the invisible microwave links, a physical mast is exactly as real a hazard for a balloon as a power tower, so a dedicated “Telecom/antenna masts” category (distinct magenta colour, own Settings checkbox) covers several plausible layer-name candidates at once, since the exact one couldn’t be confirmed from public documentation. A one-time console log now also records every layer name actually encountered that isn’t explicitly styled, to nail down the real name(s) directly from a live tile rather than guessing further.

Also found while working on this: the power-layer legend’s Tower and Transformer swatches still showed their original bright colours (yellow-orange, light blue) from before those categories were muted - the legend was never updated alongside that actual styling change, so it was silently showing the wrong colours since then. Now corrected

to match. Separately, restoring saved category selections from a stored profile now also refreshes the legend to match - it previously only updated the layer itself, so the legend could keep showing every category regardless of what had actually been saved as deselected.

Staged Descent

The Monte-Carlo landing-area scatter (run after a specific stage plan is found for a clicked target) uses antithetic sampling - each random draw is simulated together with its exact opposite - rather than fully independent draws. The wind uncertainty here is a constant bias for an entire simulated run that accumulates over the whole (potentially hours-long) path rather than averaging out step-by-step, so a small independent sample could land asymmetrically by chance and pull the drawn landing area's centroid noticeably away from the deterministic best-found plan (and so away from the intended target) rather than sitting close to it, as had been reported. Antithetic pairing guarantees the scatter stays symmetric around that deterministic path regardless of sample size.

License

Custom, source-available license in `LICENSE` (Wicki Aero GmbH) - permits running/hosting an unmodified copy, but not modification or redistribution without prior written permission. See that file for the full terms and third-party component licenses.

Airspace

The ✈ airspace layer draws real, class-filtered airspace boundaries from openAIP's Core API (confirmed working via a live CORS test - it was previously believed CORS-blocked) on top of the combined raster tile as a visual base (still the only way to show airports/navaids/reporting points, since openAIP retired separate per-category tile endpoints in 2023). The Settings checkboxes (Class A-G, Restricted, Prohibited, TMZ, RMZ) now genuinely filter which boundaries are drawn, refetched as the map moves or the selection changes. An airspace whose type/class isn't recognized by the mapping used here is shown regardless of checkbox state, rather than silently hidden.

Commercial Open-Meteo API key: optional field in Settings. When set, every Open-Meteo request in the app (wind, CAPE, gusts, elevation, ensemble) automatically switches to the dedicated `customer-api.open-meteo.com` endpoint, which has no daily call limit, instead of the free tier's 10,000/day cap - needed for genuine 24/7 unattended operation, since a single wind-profile request already counts as several calls toward that quota (any request covering more than 10 variables does). A running "Open-Meteo calls today" counter is shown in the weather model popover regardless of which tier is active.

Data sources

Source	Used for	Status
Open-Meteo	Wind profiles, elevation, CAPE, wind gusts, precipitation, ensemble members	Working. Optional commercial API key removes the free tier's daily quota.
SondeHub	Radiosonde launches, for using real sounding data instead of the	Working

	model	
api.existenz.ch (SwissMetNet)	Swiss weather stations	Working
Bright Sky	German weather stations	Working
MeteoGate/E-SOH	Weather stations across the rest of Europe (33 countries)	Station list loads; per-station value parsing not fully confirmed against a live response
ADSBExchange (via RapidAPI) + OGN/glidernet.org	Air traffic, deduplicated where both sources report the same aircraft	Working
RainViewer	Rain radar	Working
Overpass API	Airspace, terrain/land-cover, roads, region names, airports (4 mirror fallbacks: overpass-api.de, overpass.private.coffee, lz4.overpass-api.de, maps.mail.ru)	Working. The terrain check runs on its own independent queue; other features share a separate queue, so a failure in one can’t block the other. Region-limited mirrors are deliberately excluded from the fallback list - one would silently return “success, zero elements” for any position outside its coverage, risking a false-negative terrain result rather than an honest, retriable failure.
Nominatim	Reverse geocoding for landing-area place names	Working
openAIP	Airspace tiles	Working
MetarCentral	Airport METARs	Working. Airport lookup uses a curated list of major European airports first, falling back to a live Overpass query (any OSM-tagged aerodrome with an ICAO code) elsewhere.
aprs.fi	APRS station tracking	**Not functional** - CORS-blocked from the browser, confirmed with a live key. Settings clearly label this.
Xweather	Lightning (only polls when rain is detected	Confirmed working (14.08.2026): live strikes returned (HTTP 200, `success:true`). Credentials/domain fixed as described above; the `from`

	nearby)	time-range parameter was also corrected from 10 to 5 minutes, the maximum allowed without the Lightning Enterprise add-on.

Settings

Organized into color-coded groups: General (cache radius, transition altitude, units), Weather (model selection, sonde data, METAR/station sources, lightning), Air Traffic & Airspace, Terrain & Ground Features, Descent Planning (adiabatic model), Safety & Positioning (external GPS, emergency contact), and Data & Backup.

Tokens and keys: every external service credential the app uses, gathered in one place and individually editable - Open-Meteo (commercial), openAIP, RapidAPI (air traffic), aprs.fi, Xweather, EmailJS, Dropbox, GitHub. As with any client-side app, these are visible in the page source to anyone who views it; changing one asks for confirmation first.

Profile backup: export/import as a JSON file (native share sheet on mobile), or back up encrypted to a GitHub Gist (AES-GCM + PBKDF2, password-protected).

Emergency contact: pilot name, aircraft registration, mobile number, and email, with prepared-message sending over WhatsApp, SMS, or email (via mailto, or silently via EmailJS if configured).

Trajectory modeling fixes

Found the actual, deeper cause of Quick Descent's reported 180° reversal (17.08.2026), after an earlier fix (capping the delay search at 240 minutes, described below) turned out insufficient on its own: every simulation step resolved which hour's forecast wind to use via `Math.round(t/3600)` rather than `Math.floor(t/3600)`. Hourly forecast data represents the full hour starting at that mark - so 30 minutes into a flight is still within hour 0's forecast, not already hour 1's - but `round()` was switching to the next hour's forecast a full 30 minutes too early. If that next hour's forecast differs meaningfully from the current one (a realistic case, not a data error), the shown path would appear to snap onto a different wind direction partway through, exactly matching what was reported. This affected every trajectory simulation in the app (Landing Area, Quick Descent, Staged Descent, and the "wind at landing site" readouts) - all 19 occurrences of this rounding pattern are now consistently `floor()`.

Separately, fixed a genuinely broken Staged Descent: its reference trajectory (the line the staged-descent marker is dragged along) had its length tied to the "Initiate descent in" slider for consistency with Quick Descent - reasonable while that slider defaulted to 120 minutes, but once its default was later reduced to 20 minutes (in an unrelated change), the reference line became far too short to drag a marker on, search for a plan within, or show any reachable area/Monte-Carlo scatter at all. Staged Descent's reference trajectory is now a fixed 240 minutes regardless of that slider, independent of Quick Descent's own horizon.

Also fixed the earlier-described bug: Quick Descent's search for the descent-initiation delay that lands closest to a clicked target could pick a delay of up to 600 minutes (10 hours) if that happened to land closer, whenever the normal 240-minute range still looked

“improving” at its edge - an operationally unrealistic delay for an actual flight. The search is now kept within the same 0-240 minute range the “Initiate descent in” slider itself offers.

Found and fixed a further contributor to nonsensical-looking paths, particularly visible when testing without a real GPS fix: the first 1-3 minutes of every simulated cruise phase blended FROM the observed course/speed toward the forecast wind - but without a real fix yet (courseKnown still false), state.course/speedKn sit at arbitrary placeholder defaults (90°, 15kn), not an actual observation. Blending from that placeholder was pulling the very start of every path toward that arbitrary direction regardless of the real wind. With no genuine observation to blend from, the simulation now uses the actual forecast wind from t=0 instead in that situation.

Also fixed a confusing display inconsistency: the “Initiate descent in” label next to the slider only ever showed the slider’s own raw value, never updating to reflect what a Quick Descent search actually found - so it could show e.g. “20 min” right next to an “Estimated descent point” reading of “12.1 min” for the very same plan, two numbers for what looks like the same thing. The label now shows the plan’s actual found value (marked “(plan)”) while a Quick Descent plan is active, and reverts to the slider’s own value once the plan is cleared - without changing the slider’s own position, which remains Function 1’s independent setting.

Cruise-phase vector hierarchy (per explicit specification, 17.08.2026): as long as a real GPS course/speed is currently known, that vector has priority for the ENTIRE cruise-phase extrapolation - the reference trajectory is the straight-line continuation of the presently observed course/speed, for as long as that observation is available. Only when there is no GPS signal (which can also happen temporarily mid-flight, not just before the first fix) does the trajectory fall back to the forecast wind model at the current altitude instead. This replaces an earlier approach that blended from observed course toward forecast wind over just 1-3 minutes regardless of whether GPS remained available throughout - which meant even a long, fully GPS-tracked cruise phase would end up governed by the wind forecast instead of the actual observed movement almost the entire time.

Staged Descent search quality: found (in the code’s own prior comment) that the reachable-area grid resolution had been deliberately reduced in an earlier fix, reasoning that a finer grid was what made double-clicks feel unreliable - but the actual reentrancy guard already protects against that independently of resolution, so the reduction only traded away search quality without fixing anything. Too coarse a grid gave the subsequent refinement step a poor starting point, converging on a mediocre local optimum well short of the actual clicked target - producing a landing-area scatter clustered far from the target, with only the deliberately-added target vertex stretching the drawn polygon out to a thin triangle shape (matching what was reported). Resolution raised back to a genuinely fine grid.

Staged Descent reachable-area reliability: the same reentrancy guard, on a closer look, discarded an overlapping recomputation request outright rather than queuing it - since several different triggers (marker drag, various sliders) can each request a recomputation within a short window, silently dropping one left the displayed reachable area stale relative to the most recent actual state, which is very likely why it stopped showing reliably. It now remembers that another run was requested and automatically re-runs once, immediately after the current one finishes.

Known limitations

- Adiabatic braking and Monte-Carlo scatter (without a commercial API key/real ensemble data) are approximations, not calibrated against real flight data - a planning aid, not a certified instrument.

- The landing-point terrain check is an approximation, not a true line-of-sight/obstacle-clearance calculation - ring-sampled elevation plus OSM tags, since actual obstacle heights aren’t available from any free source used here.
- aprs.fi is confirmed non-functional (browser CORS restriction on their end, not fixable from this app).
- MeteoGate/E-SOH per-station value parsing is implemented but not yet confirmed against a live response.
- This is a static file with no server: after any change, it has to be re-uploaded to wherever it’s hosted before the live site reflects it. On iOS specifically, both the page itself and the service worker can be cached persistently enough that a hard reload or removing/re-adding the home-screen icon may be needed to pick up a new version.
- Cloud file pickers (Dropbox, Google Drive, etc.) shown when uploading a file are controlled by iOS/the browser based on installed provider apps, not by this page.